



POLYTECHNIC UNIVERSITY OF THE PHILIPPINES
College of Engineering
Computer Engineering Department



CMPE 103

OBJECT ORIENTED PROGRAMMING

FINAL PROJECT: IMPLEMENTATION OF LEARNING MANAGEMENT SYSTEM

Name:

Afable, Aerold D.

Bergola, Christian E.

Blanco, Eunice R.

Cruz, Kiv Zeimer C.

Cuasay,Angel Zaina

De Guzman, Irish B.

Molleda, Lirah D.

Saet, Jana Mikaela M.

Torio, Anna Sharlene A.

Torregoza, Yael C.

Section:

BSCPE 1-5

Submitted to:

ENGR. GODOFREDO "EDONG" AVENA
PROFESSOR, CMPE 103

Date Submitted:

June 15, 2026

Abstract

The Learning Management System (LMS) is a web-based application developed using Python and Tkinter to manage educational content and support the teaching and learning process. It enables instructors to create and manage courses, upload learning materials, post announcements, assign activities, and monitor students' progress. Also, it allows students to access course materials, complete assigned tasks and activities, view their grades, submit requirements, enroll in courses, and communicate with their instructors. This system aims to improve the educational experience by making learning resources more accessible and organizing academic workloads for easier management. The Learning Management System (LMS) provides an efficient, organized, and user-friendly environment that enhances learning for both instructors and students. This project demonstrates the application of object-oriented programming, implements file handling techniques, and features a graphical user interface (GUI) designed using Tkinter.

Table of Contents

Introduction.....	3
1.1 Background.....	3
1.2 Problem Statement.....	4
1.3 Project Objectives.....	4
1.4 Scope and Limitations.....	4
Methodology.....	5
2.1 Research Design.....	5
2.2 Data Collection Methods.....	5
2.3 Tools Used.....	5
System Design and Implementation.....	6
3.1 System Overview.....	6
3.2 Key Components.....	6
3.3 Implementation Process.....	6
Testing and Results.....	8
4.1 Testing Methods.....	8
4.2 Test Results.....	8
4.3 Analysis of Results.....	9
Conclusion.....	9
5.1 Summary of Findings.....	9
5.2 Recommendations.....	10
References.....	10

Introduction

1.1 Background

The Learning Management System, or popularly known as LMS in the community of higher institutions, is an online portal that connects lecturers and students. It provides an avenue for classroom materials and activities to be shared easily. It is also a portal that enables lecturers and students to interact outside the classroom through discussions and forums that could otherwise take up too much of the time intended for classroom learning. As blended learning is being implemented because of countless reasons, it is really important to have another alternative in order to continue learning despite the lack of face-to-face classes caused by changing weather conditions, lack of classrooms and facilities, health concerns such as pandemics, and other reasons. Platforms such as Learning Management Systems (LMS) are important in these types of situations. (Adzharuddin & Ling, 2013).

This study aims to create a Learning Management System that contains essential functions that help make the LMS functional and effective in improving the interaction between professors and learners while providing an effective learning experience for students. According to Jasiyah et al. (2024), a significant 63% of lecturers face difficulties when compiling and preparing instructional materials specifically for the LMS environment. Because of this problem, the proposed system prioritizes file handling to make the uploading, managing, retrieving, and downloading of instructional materials, particularly PDF lectures, less troublesome. This is one of the main issues that the study seeks to address.

Although existing LMS platforms such as Moodle and Canvas provide file-sharing and course management capabilities, users continue to experience difficulties in managing instructional materials and navigating complex interfaces (Kasim & Khalid, 2016). These challenges indicate the need for a more streamlined LMS that prioritizes efficient PDF management and a user-friendly design. While many LMS platforms offer a wide range of features, there is insufficient attention given to developing systems that focus on easing the file management such as uploading, organizing, such while maintaining ease of navigation. This gap highlights the need for a Learning Management System that emphasizes both efficient file handling and accessibility for its users.

Moreover, over the years, user feedback has consistently pointed out that the navigation of LMS platforms can be difficult to handle, leading to frustration among instructors and students. This dissatisfaction among users has often been a topic of discussion within the industry (Scheer IMC, 2024). Because of these issues, the study also aims to improve the User Experience (UX) and Graphical User Interface (GUI) of the proposed system. The interface is intended to be simple and easier to navigate in order to reduce confusion and frustration while providing a more user-friendly experience in the online learning environment.

The proposed system will provide professors with essential course management features, including the ability to create, edit, and delete courses where classes are being conducted. It will also allow professors to upload and manage PDF instructional materials directly within their courses, making learning resources easier to save, retrieve, and manage. These features can help reduce the difficulties associated with handling instructional materials while improving the overall management of courses and educational content.

For students, the system will provide the ability to browse available courses, enroll in courses, view and access enrolled courses, and unenroll when necessary. These are basic yet

essential features of a Learning Management System that allow students to manage their learning experience more effectively. Through easy access to course materials and organized course information, students can conveniently obtain the resources needed for their studies. The availability of these resources can also support continuous learning despite interruptions caused by weather conditions, lack of facilities, health concerns, and other unforeseen circumstances.

Lastly, this study aims to apply the fundamentals of Object-Oriented Programming, such as classes, inheritance, and polymorphism. These concepts can contribute to the development of a system that is organized, maintainable, reusable, and easier to improve and expand in the future.

1.2 Problem Statement

This study aims to develop a Learning Management System (LMS) that will provide an organized platform for managing courses, student enrollments, and learning materials. Specifically, it seeks to answer the following questions:

1. How can file handling techniques be implemented to support the uploading, storage, and downloading of PDF lecture materials?
2. How can a graphical user interface using Tkinter be designed and developed to provide a user-friendly experience in navigating the system?
3. How can a Learning Management System be developed to allow instructors to efficiently create, edit, and delete courses?
4. How can the system enable students to browse available courses, enroll in courses, unenroll from courses, and manage their enrollments effectively?
5. How can object-oriented programming concepts such as classes, inheritance, and polymorphism be applied in the development of the Learning Management System?

1.3 Project Objectives

The objective of this project is to create a Learning Management System. Specifically, it aims to:

- Apply object-oriented programming concepts, including classes, inheritance, and polymorphism.
- Implement file handling for saving, retrieving, and managing PDF files.
- Designed and developed with a graphical user interface using Tkinter.

1.4 Scope and Limitations

This project involves the use of Python and Tkinter in the development of a Learning Management System. The system enables instructors to create, edit, and delete courses. Moreover, it allows instructors to upload PDF learning materials associated with each course. On the other hand, students are allowed to browse available courses, enroll in courses, view their enrolled courses, and unenroll.

In addition, this project utilizes object-oriented programming concepts such as classes, inheritance, and polymorphism, along with file handling techniques for storing and retrieving course information and PDF files. Additionally, the system includes a group portfolio feature for displaying project-related information specifically the members names and assigned tasks.

The scope of the project is limited to a desktop application and focuses only on course management, enrollment management, and PDF file handling.

Methodology

2.1 Research Design

Through the process of creating the Learning Management System (LMS), it used the traditional Waterfall Software Development Life Cycle (SDLC) method as this uses a sequential approach in which each phase of development is completed before proceeding. The system has followed different stages of requirement gatherings, system analysis, system design, implementation, testing, and maintenance. During the process all the system specifications were thoroughly analyzed before finalizing, when other stages of requirements were used, the system was tested to identify if there are any errors to be fixed in the system.

The Waterfall method was adopted because it provides a more structured and organized method in developing this kind of system. It ensures that the process is well planned and completed, making it more appropriate with projects similar to the Learning Management System (LMS).

2.2 Data Collection Methods

In this project, data collection focused on gathering and analyzing the system requirements needed for the development of the Learning Management System (LMS). The requirements were gathered based on the instruction and features needed for the Learning Management System (LMS) such as adding courses, processing and managing student enrollment on their respective courses, and accessing course materials. Through the development process the system was examined to ensure that all required components were included in the system design and implementation.

Figma was also used in developing the system, using it as a tool for planning and designing, physically exploring, mapping the different interface layouts, and workflows to explore user interface layout to determine the most effective and more practical features for the system. The Figma design process also provided the base to identify both functional and non-functional requirements of the system. These requirements were used to set the frame for designing and creating the Learning Management System (LMS) prototype through Python, object-oriented programming, file handling, and Tkinter GUI. With this, different concepts on designing the system and its workflows were first visualized before development making it easier to establish a clear design requirement and add improvement to the system.

The collected requirements together with the Figma-based analysis, served as the systems foundation on designs, development, testing, and evaluation. To a large extent, the focus was on enhancing course arrangements, making tasks more visible, ensuring design consistency, and simplifying navigation, which are some of the most common usability issues in current Learning Management System (LMS) platforms. This approach ensured that the system achieved the objectives and specifications outlined in the project requirements.

2.3 Tools Used

Developing a Learning Management System (LMS) uses different types of software tools and technologies for the construction of the system. The main programming language for implementing functionalities and logic in the system was Python 3.x. GUI parts were designed

using Tkinter to build attractive and user-friendly interfaces such as forms, dashboards, and other visual components.

To perform file management operations, the system exploited `os` and suitable Python modules capable of handling directories, managing uploaded learning guides and handling student submissions. These modules helped carry out the system's file handling capabilities and data persistence.

Also, integrated development environment (IDE) and also known as Visual Studio Code (VS Code) was used for coding, debugging, and testing the application. Finally, Figma assisted the design team in coming up with interface prototypes that helped them decide on the right features and layouts for the LMS. Besides, the system designers used the prototypes as a reference for the overall system design and enhancement of usability and user experience.

System Design and Implementation

3.1 System Overview

The EduLearn Learning Management System follows a clear and simple layered dataflow by building its Graphical User Interface (GUI) using Python's Tkinter framework/library, this is the visual interface that the user sees and interact, such as logging in, viewing courses, and submitting works. These actions generate events that will trigger corresponding backend methods that checks and processes the request. Furthermore, the inputted data will be saved and retrieved from the local files stored in the computer, using formats like JSON.

3.2 Key Components

The system is designed using Object-Oriented Programming (OOP) with a base User class that contains common information such as user ID, name, username, password, email, and role, and it is extended into two subclasses: Instructor and Student. The Instructor can manage courses by adding, editing, and deleting them, as well as uploading PDF learning materials, while Students can enroll in courses, view enrolled subjects, and download PDF files. The Course class stores important details such as course name, description, assigned instructor, enrolled students, and file paths that point to locally stored PDF files using the system's directories instead of storing the files directly in the program. The system includes core features such as course management, enrollment functions, PDF upload and download, and a simple Tkinter-based graphical user interface that allows users to interact easily through forms, lists, and buttons. For data handling and system operations, the program uses modules such as `os` for file management, `json` for storing and loading data, suitable for handling file transfers like PDF uploads, `subprocess` and `sys` for system-level operations, `re` for input validation, `ssl` and `smtpplib` for secure email functionality, and `random` for generating IDs or codes. Data is stored locally using JSON, CSV, and text files so that all information is saved and can still be accessed even after the program is closed and reopened.

3.3 Implementation Process

The development of the Learning Management System (LMS) focused on keeping everything contained locally, maintaining data integrity, and separating the system logic cleanly. The application is built to run as a zero-configuration desktop application, meaning it does not require external database engines or complicated installation setups to work. Instead, it relies entirely on Python's standard libraries and separate scripts to handle data relationships and store files locally. This includes managing details like course names, descriptions, assigned instructors, enrolled students, and specific paths that point to locally stored PDF files using the system's own

directories, rather than storing the files directly inside the program. For data handling and running the system, the application uses core modules like `os` for managing files, `json` for saving and loading data, which works for handling file transfers like PDF uploads. It also uses `subprocess` and `sys` for system-level operations, `re` for validating user input, and `ssl` along with `smtplib` to handle secure email features. Everything is stored locally using JSON, CSV, and text files so that all user data is safely saved and remains accessible even after the program is closed and opened again.

One of the main issues encountered during implementation was files getting scattered across the user's computer when running dashboards from different access levels, like the instructor or student views. To keep the local storage organized, the system was refactored to eliminate broken relative path tracking and manual `ROOT_DIR` calculations inside the individual dashboard files. Instead, a centralized module called `database_for_courses.py`, which is imported everywhere simply as `db`, acts as the single source for all storage paths. By running initialization scripts through this module right when the application starts, the system automatically forces absolute path mapping that ties everything strictly to the specific local folder where the application is running. This setup ensures all critical resources stay contained. Whether a user launches the application from an automated script, an IDE terminal, or a compiled binary wrapper, structural assets like database files and session tokens are always written directly within the application's correct local workspace folder. For data persistence, the system uses lightweight, structured local JSON files where different entities are split into clean flat-file documents. When the application runs, the database module performs safety checks using automated setups so it does not accidentally overwrite anything or crash. The system looks inside the workspace folder for the `courses.json` file, and if it cannot find the file, it safely creates an empty root array(`[]`) to prevent any null pointer or missing-file errors when the user interfaces try to look up data later. At the same time, it checks the local folder structure for a folder named `file_storage`. If that folder is missing, the application tells the operating system to create it on the spot. This architecture completely separates the user interface scripts from raw storage paths, handling all data writes through a centralized data access layer instead.

To avoid performance drops and the complication of converting binary files like PDFs, documents, or slideshow presentation assets into massive data chunks inside plain-text files, a hybrid file system approach was utilized. When an instructor uploads a course asset through the graphical user interface, the system does not just read the raw file paths from the UI context. Instead, it securely copies the selected physical file directly into the local `file_storage` directory using operations from Python's standard library. At the exact same time, the actual file name is saved as a simple text string inside the matching course array entry within the JSON database. Doing it this way ensures fast file lookups and safe cross-platform file opening, while keeping a clean separation between the file's metadata and the actual physical file itself. To handle user tracking across completely separate window interfaces, the application uses a temporary, file-based session handler called `current_user.json`. Once a user enters valid credentials at the login screen, a temporary session token holding details like the full name and email address is saved locally inside the database folder. When modules like `professor.py` or `student.py` load up, they check this token file using safe try-except error handling blocks, smoothly falling back to default dummy profiles if the file is modified or if a read error happens. When a user clicks logout in the UI, the system instantly overwrites the token file with an empty dictionary, wiping out the logged-in state to keep application boundaries secure and completely closing out the session before destroying the root window.

Testing and Results

4.1 Testing Methods

Functional testing has been carried out to ensure that all functionalities in the developed LMS work as expected. Tests were carried out on vital functionalities like account creation and course enrollment. Functional Verification Testing method has been used to ensure that the functioning of the various modules of the application satisfies the criteria and requirements of the initial design specification, and hence Functional Verification Testing was used with the intention of proving the correctness of functionalities. The tests did not consider performance of the system but rather focused more on ensuring that the functionalities performed were correct, data consistent, and operations related to authentication, course records management, asset documents, password recovery, and other important tasks. It can be said that with successful running of these test cases (TC-01 – Instructor Login, TC-02 – Course Record Creation, TC-03 – File management, TC-04-TC-05 Password Recovery, TC-06 Student Registration, TC-07 Retrieval of Content, and TC-08 Withdrawal from Courses), stability and reliability of the application have been ensured.

4.2 Test Results

Test ID	Input Action Scenario	Expected System State	Status
TC-01	Instructor selects the Professor role option, enters valid account credentials, and clicks the sign-in button.	The system validates the credentials, initializes the professor session configuration, closes the login portal, and launches the instructor dashboard window.	PASS
TC-02	Instructor inputs a new course track name into the course creation field and confirms the action.	System appends the new course record information into the backend database tracking file and updates the interface list to reflect the added track.	PASS
TC-03	Instructor clicks the file attachment utility button, selects a lecture document, and uploads it to a specific course module.	The system triggers a file transfer sequence, copies the selected document cleanly into the system's local project folders, and updates the class dashboard stream for student viewing.	PASS
TC-04	User clicks the recovery link button to request assistance for a forgotten account password.	The system clears the current interface screen smoothly and loads the email verification form container.	PASS
TC-05	User submits a verified, registered email address inside the password recovery form interface.	The system generates a secure security code, sends an automated confirmation message to the recipient's email inbox, and opens the verification input field screen.	PASS

TC-06	Student selects an available class from the course catalog grid and clicks the enrollment confirmation button.	System appends the student's email profile to the course track directory, dynamically increments the "Enrolled Courses" metric tally on the main dashboard, and updates the view to switch the selection button state	PASS
TC-07	Students access an enrolled course's digital classroom interface, view the uploaded learning materials list, and click the file retrieval button to view a shared lesson document.	The system reads the document's local storage path, cross-references the operating system environment platform, launches the computer's native file viewer to display the lesson content, and flags no runtime stream errors.	PASS
TC-08	Students navigate to their active curriculum view, choose an enrolled track, and click the action button to remove themselves from the course.	The system deletes the student's email profile from that specific course data slot, recalculates the dashboard registration totals, updates the file matrix, and displays a success confirmation message box.	PASS

4.3 Analysis of Results

The test phase proved the high stability of the Learning Management System (LMS) in terms of its adherence to all functional requirements. The system was able to pass all the test cases, which were divided into the following categories: instructor authentication (TC-01), administrative course management (TC-02), and some critical functions, namely, enrollment (TC-06), retrieving content (TC-07), and withdrawing from a course (TC-08). Thus, 100% of test cases were executed successfully. Firstly, it should be noted that the work of the back-end database and the local file management system works in perfect synchronization, which makes sure that all changes are implemented accurately without any problems related to data corruption or logical mistakes. Secondly, additional test cases (password recovery – TC-04, TC-05, dynamic dashboard updating) prove efficient error-handling capabilities of the system. To conclude, the described test outcomes confirm the correctness of the application architecture, which makes the application ready for further deployment within the academic framework.

Conclusion

5.1 Summary of Findings

The findings confirmed that the Learning Management System (LMS) successfully met all the specified requirements needed in the system. All functional tests resulted in a PASS outcome, demonstrating that all major features, including user authentication, course management, file handling, password recovery, student enrollment, content access, and course withdrawal, operated correctly during use. The results highlighted the importance of the connection between the database and file management components, ensuring data consistency

and reliable system operations. Furthermore, the system demonstrated reliable data processing, accurate record management, and stable performance across all tested scenarios. Overall, the successful and testing outcomes confirm that the Learning Management System (LMS) provides an efficient, stable, and capable of managing the academic learning activities within the workspace.

5.2 Recommendations

1. Database Optimization – Change the method of storing the files from text format to structured relational database such as SQLite to ensure data integrity and avoid any data corruption issues.
2. Web Application Upgrade – Change the application’s front-end from Tkinter desktop application to an advanced framework like Django/Flask to provide better usability.
3. Quizzes and Grading System – Create a feature wherein the instructor can create quizzes and assignments and automatically grade them for the students.
4. Student Progress Tracker – Give the students visual progress tracker to monitor their learning progress.
5. Class Discussion and Announcements – Provide students a discussion forum to discuss the courses and ask any questions, while the teachers can use it to communicate any important announcements.
6. Reminders and Search Tools – Enhance navigation through course filters and automatic reminders via email or system notifications for deadlines and announcements.

References

- Adzharuddin, N. A., & Ling, L. H. (2013). Learning Management System (LMS) among university students: Does it work? *International Journal of e-Education, e-Business, e-Management and e-Learning*, 3(3), 248–252.
<https://doi.org/10.7763/IJEEEE.2013.V3.233>
- Kasim, N. N. M., & Khalid, F. (2016). Choosing the right Learning Management System (LMS) for the Higher Education Institution context: A systematic review. *International Journal of Emerging Technologies in Learning (iJET)*, 11(06), 55–61.
<https://doi.org/10.3991/ijet.v11i06.5644>
- Jasiyah, R., Rumahlewang, E., Mundung, B. I., Sairdama, S. S., & Saputra, N. (2024). Learning Management System Difficulties during the Learning Process in Higher Education. *Journal for Lesson and Learning Studies*, 7(1), 1–12.
<https://doi.org/10.23887/jlls.v7i1.68665>